

Pertinent Thoughts — Scout (530 College Basketball Pipeline)

Last synced: 2026-07-30

This document mirrors **Pertinent_Thoughts.md** (Army / OER work): important discoveries, reflections on code and results, open problems, and directions worth investigating for the **Scout** dissertation thread — ESPN box → SR advanced → player–season panel → leave-one-out teammate pool quality → draft outcome EDA.

How to use it: Add dated entries or new `##` sections as you go. Each block can follow the template: **Topic** → **Content to consider including** → **Potential placement** → **Key points** (plus optional implementation notes).

Generative Tier 1 Lab (538D, June 2026)

Topic: Soft assignment + congestion selection vs empirical 530 conditioning.

Key points:

- Thread A implemented:** `tier1_pool_assignment.py`, CELL 10–12; $\tau \approx 0.65$; Plot A calibrated to 530 CELL 8 overlap.
- Minimal score:** $S_i = A_i - wL_C$ with `crowding_smooth`; z-scored ability needs `crowding_l_z_scale` (p90–p10 of A).
- 539 vs 538D:** 539 is a **bundled DGP** (sort-chop, score noise, 90th percentile); 538D is a **modular lab** anchored to 530 — not parent/child decomposition.
- Plot B axis:** `SHOW_PLOT_B_TEAM_MEAN` — inverted-U on `team_mean`; mostly **decreasing** on L_Q LOO with same knobs.
- Status doc:** `5-Manuscript/Scout_Status_Update_for_VECTOR_Laszlo_Briefing_2026-06-02.md`.

SR–ESPN School Slugs, Small Colleges, and Coverage Equity

Topic: Unmatched schools after `bpm_merge.run_match`; manual crosswalk / alias loop; re-scrape behavior.

Content to consider including:

Many unmatched player–season rows cluster on **small religious colleges, HBCUs, NAIA-style programs, and renamed schools** where the heuristic `school_slug` (from `team_short_display_name`) does not match Sports Reference’s URL segment. The pipeline already surfaces this in `bpm_panel_rows_unmatched.csv` and optional `print_unmatched_school_lists` (aggregated by `team_id` / slug). Fixing mappings lives in `DO_NOT_ERASE/sr_school_slug_crosswalk.csv` (canonical `team_id` → `school_slug`) and `DO_NOT_ERASE/sr_school_slug_aliases.csv` (`panel_slug` → `sr_slug` for URL resolution while keeping merge keys consistent). After edits, SR refresh with `sr_refresh_do_scrape=True` only fetches **missing** (`school_slug`, `sr_year`) pairs in raw (resume logic), not a full re-scrape. **403** pairs are skipped via `bpm_scrape_skip_pairs.csv` until removed.

Potential placement:

- Data / linkage section (ESPN vs SR)
- Limitations (non-random missing SR advanced by institution type)
- Future work: conference- or success-based imputation of teammate quality where SR is absent

Key points:

- Unmatched SR is often institutional, not random — affects inference if draft-relevant talent concentrates in thin coverage cells.
- Crosswalk vs alias: crosswalk changes the canonical slug; alias fixes the URL when the panel slug should stay stable.
- Re-scrape is incremental by design; renaming a slug creates **new** job keys — old raw rows under the old slug do not automatically attach.

Ventile EDA vs Survival / CIF Framing (Officers Comparison)

Topic: What the inverted-U / binned draft-rate plots are and are not.

Content to consider including:

The 530 ventile figure bins **cross-sectional** `poolq_loo` (leave-one-out mean teammate `perf` within team–season) and plots **mean** `Y_draft` per bin vs mean `poolq_loo` — empirical draft **rate**, not time-to-event. Optional overlay is **quadratic LPM** in `poolq_loo` on the micro rows. This is **not** a cumulative incidence or survival curve; comparison to the officers workflow (CIF by bin, then bar chart of terminal values) is a **design choice** distinction worth one clear paragraph in methods. `poolq_binning` supports `quantile` (~equal n per bin; “ventiles” when `ventiles=20`) and `equal_width` on `poolq_loo`, with export slugs disambiguating runs.

Potential placement:

- Methods (nonparametric binning vs survival)
- Discussion when contrasting Scout to Army analyses
- Limitations (no time path to draft within season–year structure)

Key points:

- Each row is player–season–team; `poolq_loo` is within that season’s roster context.
- Y-axis is bin-level **draft rate**, not CIF at a horizon.
- Binning mode and z-scoring `perf` within season change interpretation of `poolq_loo` units.

Performance Measures: Box vs SR and LOO Interpretation

Topic: What enters `perf` and teammate pool quality; minutes / PPM sources.

Content to consider including:

`minutes` , `points` , `ppm` , `games` come from **ESPN game-level box** aggregated to season-team totals (PPM = total points / total minutes). **SR raw** (`bpm_player_season_raw.csv`) is the **advanced** table: `MP` , `G` , `GS` , BPM family, PER, WS, etc. — **no SR-derived PPM** in this scrape. Panel merge exposes `mp_sr` separately from box `minutes` . `perf_metric` and optional `perf_zscore_within_season` determine what is copied into `perf` before LOO; `poolq_loo` is always defined from teammates' `perf` , not from a career average across seasons.

Potential placement:

- Variable construction / data integration
- Robustness (re-run key specs across `perf_metric` choices; filenames already slugged)

Key points:

- Same athlete can contribute multiple rows (multiple seasons / teams); LOO is **within** team-season.
- SR coverage gaps leave `perf` missing for SR-backed metrics unless merged.

`min_minutes` **Floor — Ventile Sensitivity (2026-07-17)**

Topic: How the playing-time floor changes who enters the panel, who enters LOO, and whether the inverted-U survives.

Content to consider including:

With `use_prebuilt_panel_csv=False` , `min_minutes` is applied in `panel_rebuild.build_from_box` before LOO — so it defines **both** the plotted sample and the **teammate pool** used to compute `poolq_loo` . Sensitivities on the same spec (quantile bins, winsor `(0.01, 0.99)` , ppm z-scored, `restrict_teams_by_draftees=False` , 2011–2021):

<code>min_minutes</code>	Story (Charles, Jul 2026)
0	Adds ~21k mostly non-drafted deep-bench rows (82,893 vs 62k at 20); ppm and LOO noisy; ventile curve wobbly , elite dip attenuated — dilution, not falsification. Ever-drafted count barely moves (~1,136).
10	+~10.5k player-seasons vs 20; still only +2 ever-drafted rows. With 20 quantile ventiles : elite downturn spread across bins 18→19→20 (multi-bar redundancy); noisier middle ventiles; bin 20 ~1.1% draft rate.
20	Hero candidate (provisional) : rotation-level peers in LOO. With 20 quantile ventiles : rise → plateau bins 15–18 (~2.4–2.5%) → single sharp cliff at bin 20 (~0.9%); cleaner mechanism read than 10.
50 → 100	Sample collapses toward star/rotation heavy minutes only ; inverted-U largely washes out — draft rate vs poolq looks flatter; “roster pressure” variation disappears because everyone left is a high-minute, high-talent type and teammate pools homogenize among elites.

Charles observation — 10 vs 20 at 20 ventiles (2026-07-17): Same `poolq_binning='quantile'` , `ventiles=20` , `restrict_teams_by_draftees=False` . Lowering `min_minutes` from 20 → 10 does not add meaningful draft outcomes; it adds **10–19 minute** roster filler who distort LOO and reshuffle ventile ranks. **Bin 20 draft rate ~1.1% in both specs** — the **elite ventile dip is robust**; what changes is *presentation*: **10 min** = gradual 18–20 step-down (good for “redundant downturn” visuals); **20 min** = high plateau then **one** top-ventile cliff (good for “congestion in the highest peer tier” prose). Prefer **20** for Alex hero + paper main text; **10** (and **0**) in robustness appendix.

Model read (generative): If congestion bites when **comparing across peer environments** (mid vs elite LOO pools), stripping the roster down to **only stars** removes the **cross-sectional contrast** the mechanism needs — so **attenuation at high `min_minutes` is consistent with the model**, not a contradiction. Low floor adds noise; very high floor removes heterogeneity.

Potential placement:

- Methods (sample definition: rotation vs full roster vs star-only)
- Robustness appendix (`min_minutes` ∈ {0, 10, 20, 50, 100})
- Discussion linking empirical sensitivity to sim “who counts in the pool”

Key points:

- `min_minutes` is not a neutral filter — it defines the estimand and LOO algebra jointly when rebuilding from box.
- **Hero lock (provisional):** see **Hero plot candidates — three-way comparison** below — `min_minutes=20` + **16 quantile** + winsor `(0.01, 0.99)` + `use_prebuilt_panel_csv=False` .
- **High floor = star-only sensitivity:** document as “congestion signal requires roster depth variation.”
- **One-liner for Alex:** “Draft rate rises with LOO teammate quality, then drops in the **highest ventile**; dip stable at 10 and 20 minutes, washes out when we star-select or include full deep bench.”

Status: Export PNG/CSV triplet for `{0, 10, 20, 50, 100}` × `{16, 20}` quantile when archiving robustness bundle; locked hero lives in `re_entry/HEROs_and_PASSes/` .

Hero Plot Candidates — Three-Way Comparison (2026-07-17)

Topic: Charles compared three ventile EDA specs on the **same rebuilt panel** before locking the Alex side-by-side hero. Shared base for all three:

- `use_prebuilt_panel_csv=False` , `min_minutes=20` , `poolq_winsor_quantiles=(0.01, 0.99)`
- `perf_metric=ppm` , `perf_zscore_within_season=True` , `restrict_teams_by_draftees=False`
- Seasons 2011–2021 → **62,180** player-seasons, **1,134** with `Y_draft=1`
- Only fork: `poolq_binning` × `ventiles`

Content to consider including:

Candidate	Spec	Shape (Charles, Jul 2026)	Per-bin n	Verdict
1	8 equal-width	Low flat left → rise bins 4–6 (peak ~2.3–2.4%) → one drop (bin 7 ~1.1%) → bin 8 ~0	Extremely uneven — e.g. bin 3 ~22k, bin 7 ~1,145	Coarse sensitivity only; elite end = one noisy bar + empty bar
2	16 equal-width	Long climb → peak ~bin 10 (~2.5%) → several declining bars (11→16)	Wild spread — e.g. bin 8 9,712, bin 15 301, bin 16 844	Matches “≥2 bars on downturn” visually, but tail bins unreliable (see below)
3	16 quantile	Steady rise bins 2–12 → plateau bins 12–15 (~2.3–2.6%) → sharp drop only at bin 16 (~1.2%)	~3,886–3,887 every bin	Recommended hero — clearest inverted-U story, equal precision per bar

Charles observation — why 16 equal-width sends the last 2 bins to 0:

Switching from quantile to equal_width with ventiles=16 (same min_minutes=20, same winsor) does not just “zoom in” on the elite tail — it re-partitions the x-axis on poolq_loo units via pd.cut (equal intervals from min to max winsorized LOO). After z-scoring within season, most player-seasons still pile up in the middle of the LOO distribution; equal-width bins therefore get enormous n in the center and sparse n at both tails.

Exported CSV (...poolqeqwidth...2026-07-17.csv) for candidate 2:

Bin (1-indexed)	n	Mean draft rate	Mean poolq_loo
14	433	~0.23%	~0.78
15	301	0.0%	~0.91
16	844	0.0%	~1.09

So the last two equal-width bins are not empty of rows (301 + 844 = 1,145 player-seasons in bins 15–16) — but they contain zero drafted players in this run. That produces a literal cliff to 0 on the PNG, which reads like “the curve broke” rather than “congestion in the top ventile.” Why quantile bin 16 does not do this: qcut /rank quantile forces ~3,886 rows per bin, including drafted rotation players, into the top 6.25% of LOO ranks. Bin 16 still shows a dip (~1.2%, ~45 draft events expected-scale) — a statistically stable elite-tier downturn. Equal-width bins 15–16 instead isolate a tiny tail slice in LOO space where draft picks are structurally rare (~1,134 drafts total, concentrated below the extreme LOO cap).

Interpretation (not over-claiming):

- The multi-bar downturn on 16 equal-width is partly real composition (declining draft rate as mean LOO rises through bins 11–14) and partly tail sparsity — bins 15–16 hit integer zero because too few drafted players land in those narrow high-LOO intervals.
- This is not the same falsification as min_minutes=50-100 (star-only washout); it is a binning artifact that equal-width invites when poolq_loo is bell-shaped and winsor-clipped.
- Candidate 2 remains useful as appendix / sensitivity if tail bars are labeled with n (see deferred equal-width bar-width idea below) — do not treat bins 15–16 at exactly 0% as standalone proof of congestion.

Recommendation (Jul 2026 Alex prep):

Priority	Pick	Why
Hero (empirical + sim side-by-side)	Candidate 3 — 16 quantile	Rise + elite dip, equal n per bar, defensible “top ventile” language, easiest sim match
Appendix	Candidate 2 — 16 equal-width	Shows multi-bar tail decline; must annotate thin-n tail
Coarse sensitivity	Candidate 1 — 8 equal-width	Too lumpy for hero

Potential placement:

- Methods (quantile vs equal-width binning choice)
- Alex slide caption + HEROs_and_PASSes/ gallery export
- Robustness appendix (equal-width sensitivity with per-bin n)

Key points:

- poolq_binning is a presentation / estimand choice for EDA — not interchangeable with quantile ventiles when tails matter.
- Equal-width + many bins + winsorized LOO → expect zeros or near-zeros in top bins even when the quantile hero shows a clean ~1% top-ventile dip.
- Sim must match hero binning rule (16 quantile, same winsor, same min_minutes) — not just “poolq_loo on both axes.”

Status: Hero triplet + Pass A/B bundle in re_entry/HEROs_and_PASSes/; see Hero_Model_Three_Layers_Memo.md and sports/scripts/hero_model_reset_bundle.py .

High School Performance, Teammate Pools, and Binning (Advisor Discussion)

Topic: Enrich the panel with HS data; bin on pre-college ability while preserving leave-one-out **team** structure.

Content to consider including:

Discussion with advisor: add **high school** (or recruiting) data so players are **binned** along a dimension defined by **HS performance metrics** (e.g. composite rank, scouting grades, stats), rather than binning on `poolq_loo` built from **collegiate season×team** `perf` as in the current 530 ventile EDA. The conceptual **team pool** should **remain**: still compute **leave-one-out teammate pool quality** within `(team_id, season)`, but drive `perf` (or a parallel HS column) from **high-school-level measures** so “how good are my teammates *as projected from HS?*” replaces or complements “how good are my teammates *this college season on this stat?*” Implementation requires a **merge** to `athlete_id` (or a stable recruiting ID), decisions on **missing HS** coverage, and whether HS metrics are **z-scored within cohort** (class year, position) before LOO. The ventile / LPM machinery can stay the same mechanically once `poolq_loo` is defined from the chosen HS-based `perf`.

Potential placement:

- Methods (covariate timing: all pre-college information for the binning axis)
- Data section (HS / recruiting sources, linkage rate)
- Discussion (interpretation: sorting on ex-ante talent vs realized college performance)
- Limitations (selection into observable HS data; international / JUCO paths)

Key points:

- Separates **who you are surrounded by in HS terms** from **college box/SR realization** for stratification.
- LOO algebra unchanged: still exclude self within team-season; only the input to `perf` changes.
- Creates a clear story for “peer pool at arrival” vs current “peer pool on realized college metric.”
- Not yet in `sports_pipeline` — needs schema, merge QA, and sensitivity to missing HS.

Restricting to Teams With at Least One Drafted Player

Topic: Drop rows for teams with **no** drafted player under a clear rule (program vs season).

Content to consider including:

Implemented in code (`panel_build.filter_panel`): `restrict_teams_by_draftees` (default **True**) and `draftee_restriction`: `all_time` — keep only `team_id` with ≥ 1 row with `Y_draft==1` anywhere in the filtered sample before this step; `season` — keep only `(team_id, season)` where ≥ 1 roster member has `Y_draft==1` that season. Applied after `dropna(poolq_loo, Y_draft)` and `min_minutes`, so ventiles, LPM, and integrity **use** all align. Export slugs gain `_tdalltime` or `_tdseason` when the restriction is on (see `perf_metric.export_plot_slug`). Motivation: focus draft-rate vs pool-quality on environments where draft outcomes are not structurally zero on the roster. Trade-offs: smaller sample, selection (e.g. low-majors), changed interpretation — report **counts dropped** and run **robustness** with `restrict_teams_by_draftees=False`.

Potential placement:

- Sample definition / inclusion rules
- Robustness appendix (full sample vs draft-exposed-teams-only)
- Limitations (generalization beyond teams that ever produce a draft pick)

Key points:

- Sharpens comparison sets where draft outcome is empirically possible on the roster.
- Risks truncating heterogeneity — justify and show robustness.
- `all_time` vs `season` answers “program ever had a draftee in window” vs “this season’s roster had a draftee.”

Charles observation (2026-07-17): Restricting to teams who have **ever** had a draftee (`restrict_teams_by_draftees=True`, `draftee_restriction="all_time"`) produces **startlingly different** ventile curves vs the full sample (`restrict_teams_by_draftees=False`). The shift is **even larger** with `draftee_restriction="season"` — keep only `(team_id, season)` rosters that actually had a draft pick that year. Treat this as a **major sample-definition fork**, not a minor robustness tweak: the inverted-U shape, tail bins, and draft rates can move a lot. **Alex hero plot** currently uses `restrict_teams_by_draftees=False`; any draftee-restricted run is a **different estimand** and must be labeled explicitly in provenance and side-by-side exports.

Status: Sensitivity branch — compare full sample vs `all_time` vs `season` in appendix; do not silently swap for the canonical empirical curve without noting it.

Sorting Noise Robustness Thought

Topic: Later robustness version for noisy assortative sorting in `537_Sports_Simulation.ipynb`.

Content to consider including:

For the first noisy-sorting implementation, use raw Gaussian sorting noise:

`sorting_signal_i = A_i + epsilon_i`, where `epsilon_i ~ Normal(0, SORTING_NOISE_SD)`.

This keeps the first version easy to explain: true ability `A_i` is unchanged, but the pool assignment process observes a noisy version of ability. When `SORTING_NOISE_SD = 0`, sorting is perfectly assortative; as it increases, pool assignment becomes less perfectly ordered.

Later, revisit a relative-noise version:

`epsilon_sd = SORTING_NOISE_FRACTION * sd(A_i)`.

That version may be more portable across different ability distributions because the noise scale adjusts to the observed spread of ability. Hold this for a robustness/sensitivity branch after the raw-noise version is clear.

Potential placement:

- Simulation appendix or robustness subsection
- Notes around noisy sorting in `537_Sports_Simulation.ipynb`

Key points:

- Noise is **only for sorting into pools**, not for true ability or promotion weights.
- Raw `SORTING_NOISE_SD` is easier to teach first.
- Relative `SORTING_NOISE_FRACTION * sd(A_i)` is the later robustness idea to remember.

Equal-Width Ventile Bars: Encode Bin n Visually (2026-07-17)

Topic: When `poolq_binning="equal_width"`, bar counts per bin are unequal (unlike quantile/`qcut` ventiles). The draft-rate height alone can misread precision in thin tail bins.

Content to consider including:

For `ventile_eda_plot_style="bins_bars_520"` (530 CELL 4 / `panel_build.ventile_plot`), optionally encode `n` from the binned table (`ventile_table` → column `n`) in the bar geometry or color — e.g. **narrower or lighter bars** when a bin has fewer player-seasons, **full width / saturated color** when `n` is large. Goal: reader sees at a glance that equal **x-width** ≠ equal **sample size** (especially bins 1–2 and 14–16 on the current 16-bin ppm spec).

Potential placement:

- 530 ventile PNG polish (methods figure or appendix)
- Same pattern for generative side-by-side plots once sim bins are exported with counts

Key points:

- **Deferred** — do not block Alex side-by-side / `poolq_loo` sim work (Jul 2026).
- Quantile binning already approximates equal `n`; this idea matters mainly for **equal_width** runs.
- CSV already has per-bin `n`; change is plotting-only in `ventile_plot`.

Status: Idea logged; not implemented.

Equal-Width Ventile Bars: Encode Bin n Visually (2026-07-17)

Topic: When `poolq_binning="equal_width"`, bar counts per bin are unequal (unlike quantile/`qcut` ventiles). The draft-rate height alone can misread precision in thin tail bins.

Content to consider including:

For `ventile_eda_plot_style="bins_bars_520"` (530 CELL 4 / `panel_build.ventile_plot`), optionally encode `n` from the binned table (`ventile_table` → column `n`) in the bar geometry or color — e.g. **narrower or lighter bars** when a bin has fewer player-seasons, **full width / saturated color** when `n` is large. Goal: reader sees at a glance that equal **x-width** ≠ equal **sample size** (especially bins 1–2 and 14–16 on the current 16-bin ppm spec).

Potential placement:

- 530 ventile PNG polish (methods figure or appendix)
- Same pattern for generative side-by-side plots once sim bins are exported with counts

Key points:

- **Deferred** — do not block Alex side-by-side / `poolq_loo` sim work (Jul 2026).
- Quantile binning already approximates equal `n`; this idea matters mainly for **equal_width** runs.
- CSV already has per-bin `n`; change is plotting-only in `ventile_plot`.

Status: Idea logged; not implemented.

Alex Follow-Up — Pass A/B Approved; Three Empirical Frontiers (2026-07-30)

Topic: After Alex review of Pass A (λ knockout) and Pass B (ρ ablation), both parties are satisfied with the generative diagnostics. Alex wants to push from “mechanism exists in sim” toward **empirical identification** and **real-world magnitude**.

Context (Charles ↔ Alex, Jul 2026):

- Pass A and Pass B results are **approved** — proceed without re-litigating those bundles.
- Next questions are not “does the code work?” but “where does this show up in data, and how big is it for real people?”

1. Random pools ($\rho \approx 0$) but L_Q still exists — Hero or Naïve?

Question: Are there environments where peer pools / teams are **essentially randomly formed** (sim’s $\rho = 0$ story), yet we can still compute leave-one-out pool quality L_Q (`poolq_loo`)? If so, do we see the **Hero** (inverted-U on L_Q) or the **Naïve** (monotone “better peers → better outcomes”)?

Content to consider including:

- **Generative anchor:** Pass B already holds score fixed and sweeps ρ ; $\rho = 0$ arm is the “max mixing” benchmark. Empirical analogue needs quasi-random seating with observable rosters.
- **Candidate domains (not all in current repo):**
 - **Army — Ranger School (Charles, Jul 2026):** Strong **B and D** environment (stress, sleep deprivation, peer comparison — real $L_{\text{net}} = B - D$ story), but trainees are **randomly assigned to training platoons** at the start. That is a credible $\rho \approx 0$ seating draw with observable rosters: you can still compute leave-one-out peer quality within each platoon, then ask whether outcome (e.g. completion, honors,

recycle) vs L_Q looks **Hero** or **Naïve**. Good cross-domain foil to sorted MBB rosters — verify assignment rules and what “outcome” means in the data when AWS access returns.

- o **Army (general)**: other entry cohorts assigned to units — may *not* be $p = 0$; Ranger is the cleaner random-platoon example.
- o **Academia / housing**: classic **random roommate** designs (Harvard-style) — L_Q -like peer quality exists; assignment plausibly random at dorm draw.
- o **Basketball (hard)**: college rosters are **highly sorted** (recruiting, conferences). True $p \approx 0$ is rare. Proxies only: early walk-on cohorts, some camp/all-star **random team** draws, or **within-team** subsamples where peer mix is exogenous to individual talent (weak).
- **What to plot**: Same hero estimand — mean outcome by **quantile bins on L_Q** — in the random-assignment subsample only. Compare curve shape to full MBB hero and to **Naïve** (monotone) prediction.
- **Sharp prediction if congestion-in-score matters**: Under $p \approx 0$, **elite- L_Q compression / inverted-U dip should weaken or disappear** even though L_Q is still defined (Pass A/B logic: sorting + congestion-in-score interact).

Potential placement:

- Cross-domain methods (identification paragraph)
- Basketball limitations + Army / tenure / roommate literature bridge
- New empirical subsection: “environments with random peer assignment”

Key points:

- This is a **falsification / identification** test, not a replication of the MBB hero on the same sample.
- **Data acquisition** may dominate science — flag which domains Charles can actually access (Army AWS paused; MBB may not have a clean $p = 0$ arm).
- Even a **negative result** (“we found no setting with both random pools and stable L_Q ”) is publishable honesty.

Status: Open — scout data sources; no pipeline change yet.

2. Normal assortativity ($p > 0$) but no L_Q or $\lambda \approx 0$ — what should we see?

Question: Are there settings with **usual sorting** into peer groups, but either (a) **no meaningful L_Q estimand**, or (b) **$\lambda \approx 0$** so advancement/scoring ignores congestion? What does the outcome curve look like?

Content to consider including:

- **Disambiguate two sub-cases Alex may mean**:
 1. **Sorted pools, $\lambda = 0$ (talent-only score / selection)**: Generative Pass A **talent-only arm** — monotone rise on **poolq_loo**, **no elite dip**. Empirical analogue: domains where promotion is **explicitly merit-only** on individual metrics (some contests, combine-only cuts?) while peers still cluster by ability because of sorting.
 2. **Sorted pools, “no L_Q ” channel**: Outcome depends on **individual A_i** only; peer quality is **measured** but **causally irrelevant** to the selection rule. Prediction: **Naïve on ability; flat or noisy** relationship on L_Q bins (no inverted-U).
- **Basketball partial analogue**: Outcomes tied to **individual NBA combine / draft stock** measures that ignore college peer context; plot draft success vs **poolq_loo** may flatten when conditioning on strong individual ability controls.
- **Contrast with Hero**: Same assortativity ($p > 0$) **plus** congestion in score ($\lambda > 0$) $\rightarrow$ inverted-U on L_Q (MBB hero + Pass A congestion arm).

Potential placement:

- Discussion: when the mechanism should **not** apply
- Methods: define $\lambda = 0$ vs “ L_Q not in selection rule” operationally
- Cross-domain table: (p , λ , expected curve shape)

Key points:

- Pass A already gives the **$\lambda \approx 0$** generative picture; empirical hunt is for a **real domain** that matches that arm.
- “No L_Q ” is easy to garble — usually we mean **L_Q is not in the advancement channel**, not that teammates don’t exist.

Status: Open — conceptual clarity + candidate case studies per domain.

3. Real-world magnitude — predictive importance of roster pressure (Paper Directions 14)

Question (Alex, 2026-07-30): Ability alone predicts promotion/draft well. **How much does adding roster pressure improve prediction** — vs a model that uses ability only? Is the gain ~10% overall? **10–20% more for top performers?** Or a rounding error when the prediction that matters (“will I get promoted?”) is essentially unchanged?

Source transcript: [transcripts/20260730-Paper_Directions_14_otter_ai_transcript.docx](#)

Action spec (one page): [3-Master_Plan/re_entry/05_Alex_Magnitude_Spec.md](#)

What Alex is NOT asking:

- Overlay Hero ventile curve on Naïve ability curve and call it done — Hero is **$E[Y \mid \text{poolq_loo}]$ with roster pressure already in the world**.
- Rewind development (“what if you hadn’t played four years on that team?”) — ppm/ability may reflect roster context; Alex accepts a **fabricated assessment** counterfactual instead.

What Alex IS asking:

1. **Fit Model A (full)**: **$Y_{\text{draft}} \sim \text{ability} + \text{poolq_loo} + \text{poolq_loo}^2$** (roster in selection score). Alex: **“You have to give me a fit.”**
2. **Fit Model B (ability-only)**: **$Y_{\text{draft}} \sim \text{ability}$** — selectors ignore roster congestion ($\lambda = 0$ in **prediction**; unlimited scout capacity story).
3. **Compare per-person $\hat{p}_i^{\text{full}}$ vs $\hat{p}_i^{\text{ability}}$** — how big is $|\Delta_i|$?
4. **Overall predictive gain**: ΔAUC , Brier, or “~X% predictive capacity” — define denominator.
5. **Ability-dependent error**: Alex’s hypothesis — **overall gain may look modest, but $|\Delta|$ largest at top ability** (elite peer congestion bites there).

6. **Carrying capacity (K)**: Few slots (NBA ~60 picks) → roster pressure may **drive** predictions; many slots → rounding error. Cross-domain later.
- Charles ↔ Alex dialogue (clarifying moment)**:
- Charles: “Isn’t the inverted-U the quantification?” → Alex: **Partially**, but still computed **with** roster pressure present; need **vs no roster in the model**.
 - Charles: “Difference between two curves?” → Alex: **Predictive accuracy** — “How much better are you doing by knowing the roster?”
 - Army analogy (Charles): no cap on top blocks → fair individual assessment. Alex: **Right** — that’s the counterfactual **selection score**, not undoing benefits.
- Charles side idea (parked)**: “Team player” / plays-well-with-others — Alex likes intuition, **too many confounders** for now.
- Content to consider including (empirical, basketball-first)**:
- Locked hero panel (530); same ability metric as hero (ppm z within season).
 - Export: model comparison text + CSV of predicted probs + optional `|Δ|` by ability ventile PNG → `HEROs_and_PASSes/MAGNITUDE_*` when script exists.
 - Generative link: Pass A talent-only arm = **sim** $\lambda = 0$; this section = **empirical** Model B.
- Potential placement**:
- Results: predictive comparison subsection
 - Alex packet: one slide “roster pressure and prediction accuracy”
 - Discussion: when effect is large vs rounding error; role of K
- Key points**:
- Most important** Alex frontier after Pass A/B approval — **predictive importance**, not only curve shape.
 - Hero LPM on `poolq_loo` alone is **not** the full deliverable; need **Model A vs B** on micro rows.
 - Say out loud: fabricated assessment world; ability may be roster-contaminated; report **n** by stratum.
- Status**: Spec written (`05_Alex_Magnitude_Spec.md`); **analysis not run** — checklist §5.
-

Small Things to Check Later

- Confirm whether `Y_draft` should be “ever drafted” vs draft **in or after** season *k* for causal timing stories.
- Document any manual edits to `sr_school_slug_crosswalk.csv` in a one-line changelog row or git commit message when possible.
- `export_panel_after_run` during multi-metric sweeps overwrites one path — document if a dated archive is needed.
- Implement and document **HS → athlete** merge keys; add `poolq_loo_hs` (or swap `perf` source) when data land.
- Log **N excluded** by `restrict_teams_by_draftees` in a one-off QA cell or export (optional enhancement).